

EECE7352 Computer Architecture

Homework 4

Sakthivel P. Sivakumar

NUID: 002312294

Part D: (15 Points)

Read that the attached paper on using deep learning to guide cache replacement. Describe how an LSTM model can be used to guide replacement. Why is this type of replacement scheme more effective than LRU? Explain the fundamental differences between heuristic-driven versus learning-based schemes. Even though this paper was published in 2019, there has been more recent work that cites the ideas from this paper (the Glider paper has been published over 100 times). Describe the work that has cited this paper and discuss how it differs from Glider. Make sure to discuss at least two more recent papers that cite the Glider paper.

Paper Summary:

This research paper, titled “**Applying Deep Learning to the Cache Replacement Problem**” introduces a novel hybrid methodology to create a new, state-of-the-art cache replacement policy called **Glider**. The authors first use a powerful, unconstrained Long Short-Term Memory (LSTM) neural network in an offline setting to analyse and learn the characteristics of optimal cache replacement. By interpreting this complex model, they extract a key insight about program behaviour, which then allows them to design Glider, a simple, hardware-friendly online predictor that outperforms previous policies.

Q1: how an LSTM model can be used to guide replacement?

The paper’s goal was to prove that a deep learning model *can outperform existing predictors*. For this, the authors frame cache replacement as a sequence labelling problem.

- **Model Input:** The model doesn't look at data addresses and instead it analyses a sequence of Program Counters and unique instruction locations that load data— to predict the caching decision.
- **Training Labels:** The LSTM is trained offline using **Belady's MIN algorithm** as the "oracle". This means it learns by trying to replicate the decisions of the provably optimal replacement policy but works only in offline.
- **Prediction:** For each new PC in a sequence, the model outputs a **binary label**: "cache-friendly" (high priority) or "cache-averse" (low priority).

The attention-based LSTM model proved highly effective, achieving **82.6% accuracy** in predicting optimal decisions, a significant jump from the 72.2% accuracy of Hawkeye, the previous state-of-the-art.

The major reason for this success is its ability to **Capture Long-Range Context**, as the LSTM effectively uses a history of approximately 30 PCs to make its decision, capturing complex, long-range dependencies in a program's control flow.

Another reason is the ability to learn **program semantics**; The model learns to associate specific PC sequences with caching behaviour.

Q2: Why is this type of replacement scheme more effective than LRU?

The reason why LSTM based approach was better than LRU is because of its **adaptability and context awareness**.

The LRU is a simple, **non-adaptive heuristic**. It assumes all recently accessed data is important, which fails in many common scenarios such as streaming, scanning etc

Whereas an **LSTM** based approach **can distinguish access types** by identifying that a PC associated with a streaming scan is "cache-averse," while a PC associated with a pointer-chase is "cache-friendly". Additionally, the model **learns to make decisions** that are provably close to optimal, rather than just following a fixed rule that may or may not work.

Q3: Explain the fundamental differences between heuristic-driven versus learning-based schemes?

Heuristic-driven schemes are static and use fixed, predefined rules. The designers create these rules based on assumed common access patterns, like LRU assuming temporal locality. The same rule is applied to all programs, whether it's effective for that program's specific behaviour or not.

On the other hand, the Learning based schemes are dynamic and adaptive. They don't rely on fixed assumptions. Instead, they discover program specific patterns by analysing execution data like PC sequences. They learn which behaviours predict a cache hit or miss for that specific program and adjust their decisions accordingly.

Q4: Describe the work that has cited this paper and discuss how it differs from Glider?

For my comparison with the Glider paper, I took the Pond paper, it describes the first memory pooling system that both meets cloud performance goals and significantly reduces DRAM cost.

Glider applies offline deep learning to discover program-context features that improve on-chip cache replacement decisions, whereas Pond uses machine learning at the datacentre scale to predict VM memory sensitivity and untouched memory for CXL-based DRAM pooling. While Glider optimizes microarchitectural cache behaviour, Pond focuses on reducing cloud DRAM costs while maintaining NUMA-local performance, operating at a fundamentally different layer of the memory hierarchy.

Citation: (APA)

Li, H., Berger, D. S., Hsu, L., Ernst, D., Zardoshti, P., Novakovic, S., ... & Bianchini, R. (2023, January). Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (pp. 574-587).

Q5: discuss at least two more recent papers that cite the Glider paper?

Citation 1: (APA)

Mostofi, S., Gupta, S., Hassani, A., Tibrewala, K., Teran, E., Gratz, P. V., & Jiménez, D. A. (2025, June). Light-weight Cache Replacement for Instruction Heavy Workloads. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture* (pp. 1005-1019).

Summary:

This paper proposes a lightweight, instruction-focused cache replacement policy designed specifically for workloads dominated by instruction fetches. The paper introduces a mechanism that tracks instruction access behaviour with minimal hardware cost, identifying persistent instruction blocks and evicting those that are unlikely to be reused. By reducing complexity and overhead compared to ML-based or history-rich replacement schemes, the policy achieves better miss rates and IPC for instruction-heavy applications, making it attractive for processors with tight power or area budgets.

Citation 2: (APA)

Xing, S., & Wang, Y. (2025). Proactive data placement in heterogeneous storage systems via predictive multi-objective reinforcement learning. *IEEE Access*.

Summary:

This paper introduces a proactive storage tiering framework that uses deep learning [LSTM + Transformers] to predict future data accesses and a multi-objective reinforcement learning agent to place data across heterogeneous storage tiers. Unlike traditional reactive LRU or heat-based tiering, the system anticipates future access patterns, optimizing latency, throughput, and cost simultaneously. Experiments show substantial improvements—up to 45% lower latency and 28% lower cost demonstrating the power of combining predictive modelling with RL for large-scale storage optimization.